

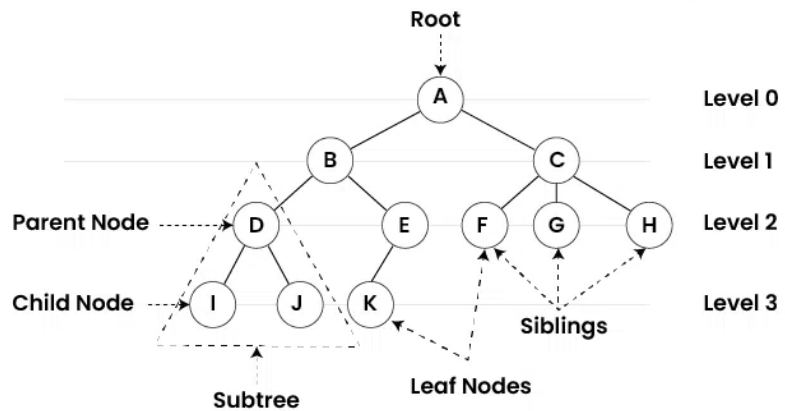
Tree Data Structure

Last Updated : 12 May, 2025



Tree Data Structure is a non-linear data structure in which a collection of elements known as nodes are connected to each other via edges such that there exists exactly one path between any two nodes.

Tree Data Structure



Basics of Tree Data Structure

- [Introduction to Tree](#)
- [Types of Trees in Data Structures](#)
- [Applications of tree data structure](#)

Binary Tree

1. [Binary Tree Guide](#)
2. [Applications and Advantages of Binary Tree](#)
3. [Properties of Binary Tree](#)
4. [Types of Binary Tree](#)
5. [Inorder Traversal of Binary Tree](#)
6. [Preorder Traversal of Binary Tree](#)
7. [Postorder Traversal of Binary Tree](#)
8. [Level Order Tree Traversal](#)
9. [Height or Depth of a Binary Tree](#)
10. [Height and Depth of Node](#)

11. [Level of a Given Node in Tree](#)
12. [Search a Node in Binary Tree](#)
13. [Find the Parent in Binary Tree](#)
14. [Insertion in a Binary Tree](#)
15. [Deletion in a Binary Tree](#)
16. [Enumeration of Binary Trees](#)
17. [Find all Leaf nodes](#)
18. [Binary Tree \(Array implementation\)](#)

Binary Search Tree

1. [Binary Search Tree Guide](#)
2. [Applications of BST](#)
3. [Insertion in Binary Search Tree](#)
4. [Searching in Binary Search Tree](#)
5. [Binary Search Tree \(BST\) Traversals – Inorder, Preorder, Post Order](#)
6. [Deletion in Binary Search Tree](#)
7. [More Problems on Binary Search Tree](#)

AVL Tree

1. [AVL Tree Data Structure](#)
2. [Insertion in an AVL Tree](#)
3. [Deletion in an AVL Tree](#)
4. [Weak AVL or Rank Balanced Trees](#)
5. [Insertion, Searching and Deletion in AVL trees containing a parent node pointer](#)
6. [AVL with duplicate keys](#)
7. [Count greater nodes in AVL tree](#)
8. [How to insert Strings into an AVL Tree](#)
9. [Minimum number of nodes in an AVL Tree with given height](#)
10. [Optimal sequence for AVL tree insertion \(without any rotations\)](#)
11. [Different shapes of AVL possible at height h](#)

Red-Black Tree

1. [Introduction to Red-Black Tree](#)

2. [Insertion in Red-Black Tree](#)
3. [Red-Black Trees | Top-Down Insertion](#)
4. [Deletion in Red-Black Tree](#)
5. [Applications, Advantages, and Disadvantages of Red-Black Tree](#)

Ternary Search Tree

1. [Ternary Search Tree](#)
2. [Ternary Search Tree meaning & definition in DSA](#)
3. [Ternary Search Tree \(Deletion\)](#)
4. [How to implement text Auto-complete feature using Ternary Search Tree](#)
5. [Longest word in ternary search tree](#)

n-ary or Generic Tree

1. [Generic Trees\(N-ary Trees\)](#)
2. [What is Generic Tree or N-ary Tree](#)
3. [Depth of an N-ary Tree](#)
4. [Children of a Node in an N-ary Tree](#)
5. [Find Siblings in an N-ary Tree](#)
6. [Mirror of n-ary Tree](#)
7. [Diameter of an N-ary Tree](#)
8. [Level Order Traversal of N-ary Tree](#)
9. [Sum of all elements of N-ary Tree](#)
10. [Serialize and Deserialize an N-ary Tree](#)

B Tree

1. [Introduction of B-Tree](#)
2. [What is B-Tree? | B-Tree meaning](#)
3. [Insert Operation in B-Tree](#)
4. [Delete Operation in B-Tree](#)
5. [B-Tree Insert without aggressive splitting](#)

B+ Tree

1. [Introduction of B+ Tree](#)

2. [What is B+ Tree | B+ Tree meaning](#)
3. [Insertion in a B+ tree](#)
4. [Deletion in B+ Tree](#)

Other types of Trees

- **Ternary Tree:**
 1. [What is Ternary Tree](#)
 2. [Create a Doubly Linked List from a Ternary Tree](#)
- **Interval Tree:**
 1. [Interval Tree](#)
 2. [Interval Tree using GNU Tree-based container](#)
- **2-3-4 Tree:**
 1. [2-3-4 Tree](#)
 2. [2-3 Trees |\(Search, Insert, and Deletion\)](#)

Trees vs other Data Structures

1. [Difference between graph and tree](#)
2. [Comparison between Heap and Tree](#)
3. [What is the difference between Heap and Red-Black Tree?](#)
4. [Difference between Binary Search Tree and Binary Heap](#)
5. [Difference between Stack and Tree](#)
6. [Difference between an array and a tree](#)

Comparison among different Tree Data Structures

1. [Difference between General tree and Binary tree](#)
2. [Difference between Binary Tree and Binary Search Tree](#)
3. [Difference between Binary tree and B-tree](#)
4. [Difference between B tree and B+ tree](#)
5. [Difference between Full and Complete Binary Tree](#)
6. [Difference between Binary Search Tree and AVL Tree](#)
7. [Red Black Tree vs AVL Tree](#)

Problems based on Tree Data Structure

Problems	Difficulty Level	Solve
Height of Binary Tree	Easy	Solve
Determine if two trees are identical	Easy	Solve
Mirror tree	Easy	Solve
Symmetric Tree	Easy	Solve
Diameter of tree	Easy	Solve
Checked for Balanced tree	Easy	Solve
Children Sum Parent	Easy	Solve
Check for BST	Easy	Solve
Array to BST	Easy	Solve
Largest value in each level of binary tree	Easy	Solve
Maximum GCD of siblings of a binary tree	Easy	Solve
Zigzag Tree Traversal	Easy	Solve
Inorder Successor in BST	Easy	Solve

Problems	Difficulty Level	Solve
Kth Largest Element in a BST	Easy	Solve
Check if subtree	Medium	Solve
Single Valued Subtree	Medium	Solve
Unique BSTs	Medium	Solve
Inorder Traversal (iterative).	Medium	Solve
Preorder Traversal (iterative).	Medium	Solve
Postorder Traversal(iterative).	Medium	Solve
Vertical Traversal of a Binary Tree	Medium	Solve
Boundary Traversal	Medium	Solve
Construct Binary Tree from Parent array.	Medium	Solve
Construct Binary Tree from Preorder and Inorder Traversal	Medium	Solve
Preorder Traversal and BST	Medium	Solve
Construct tree from preorder traversal	Medium	Solve

Problems	Difficulty Level	Solve
Minimum distance between two given nodes	Medium	Solve
Maximum sum leaf to root path	Medium	Solve
Odd Even Level Difference	Medium	Solve
Lowest Common Ancestor of a Binary Tree	Medium	Solve
Ancestors in Binary Tree	Medium	Solve
Remove BST keys outside the given range	Medium	Solve
Pair with given target in BST	Medium	Solve
Sum Tree	Medium	Solve
BST to greater sum tree	Medium	Solve
BST to max heap	Medium	Solve
Clone binary tree with random pointer	Medium	Solve
Maximum sum of non adjacent nodes	Medium	Solve
Largest BST in a Binary Tree	Medium	Solve

Problems	Difficulty Level	Solve
Extreme nodes in alternate order	Medium	Solve
Connect nodes at same level	Hard	Solve
Nodes at given distance in a Binary Tree	Hard	Solve
Sorted Linked List to BST	Hard	Solve
Binary Tree to Doubly Linked List	Hard	Solve
Maximum sum path between two leaf nodes	Hard	Solve
K-Sum Paths	Hard	Solve
Number of turns in a binary tree	Hard	Solve
Merge two BST's	Hard	Solve
Fixing two nodes of a BST	Hard	Solve
Burn Binary Tree	Hard	Solve

Quick Links:

- [Practice problems on Trees](#)
- [Quizzes on Tree](#)

[Comment](#)[More info](#) [Advertise with us](#)[Next Article](#) [Heap Data Structure](#)

Similar Reads

DSA Tutorial - Learn Data Structures and Algorithms

DSA (Data Structures and Algorithms) is the study of organizing data efficiently using data structures like arrays, stacks, and trees, paired with step-by-step procedures (or algorithms) to solve problems...

 7 min read

Array Data Structure Guide

In this article, we introduce array, implementation in different popular languages, its basic operations and commonly seen problems / interview questions. An array stores items (in case of C/C++ and Java Primiti...

 4 min read

String in Data Structure

A string is a sequence of characters. The following facts make string an interesting data structure. Small set of elements. Unlike normal array, strings typically have smaller set of items. For example, lowercase...

 3 min read

Matrix Data Structure

Matrix Data Structure is a two-dimensional array arranged in rows and columns. It is commonly used to represent mathematical matrices and is fundamental in various fields like mathematics, computer...

 2 min read

Searching Algorithms

Searching algorithms are essential tools in computer science used to locate specific items within a collection of data. In this tutorial, we are mainly going to focus upon searching in an array. When we...

 3 min read

Sorting Algorithms

A Sorting Algorithm is used to rearrange a given array or list of elements in an order. For example, a given array [10, 20, 5, 2] becomes [2, 5, 10, 20] after sorting in increasing order and becomes [20, 10, 5, ...

 3 min read

Hashing in Data Structure

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. Hashing involves mapping data to a specific index in a hash table (an array of...

🕒 3 min read

Two Pointers Technique

Two pointers is really an easy and effective technique that is typically used for Two Sum in Sorted Arrays, Closest Two Sum, Three Sum, Four Sum, Trapping Rain Water and many other popular interview...

🕒 11 min read

Sliding Window Technique

Sliding Window Technique is a method used to solve problems that involve subarray or substring or window. The main idea is to use the results of previous window to do computations for the next window...

🕒 13 min read

Prefix Sum Array - Implementation and Applications

Given an array `arr[]` of size `n`, the task is to find the prefix sum of the array. A prefix sum array is another array `prefixSum[]` of the same size, such that `prefixSum[i]` is `arr[0] + arr[1] + arr[2] . . . arr[i]`. Examples:...

🕒 8 min read
